Inventory & Order Management API Enhancement Report

Name: Al-Jolanda Al-Handhali

Date: 25 June 2026

Abstract

This report presents the enhancement of an existing Inventory & Order Management API by improving its database design, relational mapping, and software implementation. The project involved redesigning the Entity Relationship Diagram (ERD) to introduce complex and multivalued attributes, adding new entities such as Order Item, Customer Phone, and Order Status History, and refining relationships between existing entities. Based on the updated design, the relational mapping and application layers were modified accordingly. New entity classes, repositories, services, and REST API endpoints were implemented to support enhanced functionality. Additionally, JSON serialization issues caused by circular references were resolved using appropriate Jackson annotations. These enhancements resulted in a more normalized database structure, improved maintainability, better support for business requirements, and a more scalable REST API architecture.

Contents

Table of Figures

1. Introduction

The purpose of this project was to enhance an existing **Inventory & Order Management REST API** by improving its database design, relational mapping, and software implementation. The original project provided the core functionality for managing customers, products, categories, inventory, and orders. However, several improvements were required to better represent business requirements, increase maintainability, and extend the application's functionality.

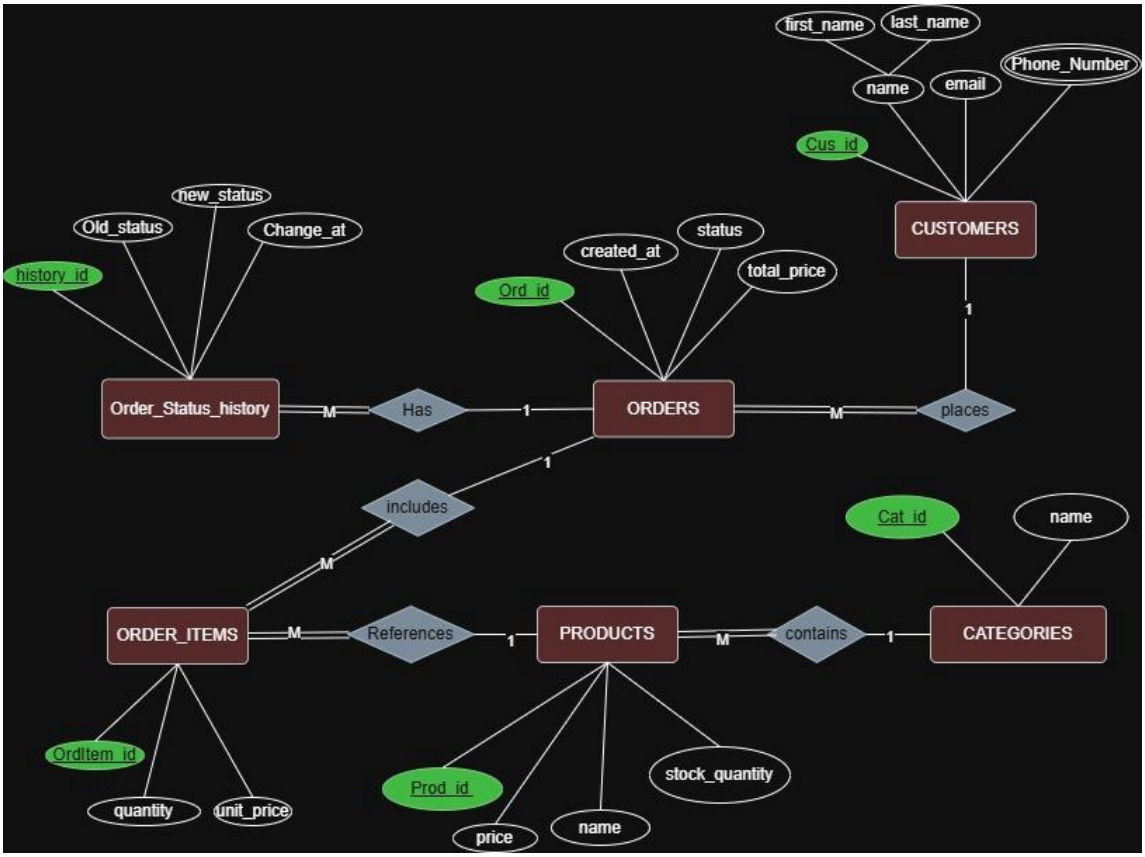
The enhancement process included redesigning the Entity Relationship Diagram (ERD), updating the relational mapping, introducing new entities and repositories, extending the service and controller layers, and resolving serialization issues within the REST API.

2. Project Enhancements

The project was enhanced in three major areas:

2.1 ERD Improvements

Figure 1: Enhanced Entity Relationship Diagram



The enhanced ERD introduces the Customer Phone, Order Item, and Order Status History entities. It also improves relationships, participation constraints, and customer attributes to better represent the business requirements.

The conceptual database design was refined by introducing additional entities and improving existing relationships.

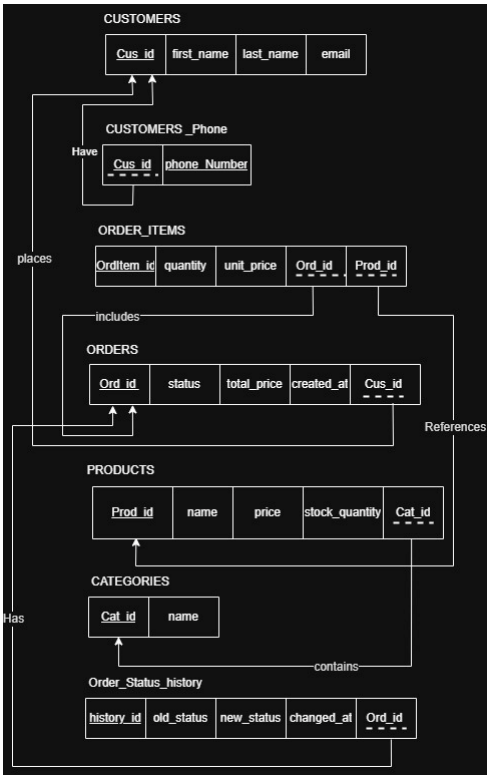
The following enhancements were made:

- Replaced the Customer **name** attribute with **firstName** and **lastName** as a complex attribute.
- Added **phone number** as a multivalued attribute for Customer.
- Added the **Order Item** entity.
- Added the **Order Status History** entity.
- Improved relationships between entities.
- Refined participation constraints to better represent business rules.

These changes resulted in a completer and more complete and more normalized conceptual model.

2.2 Relational Mapping Improvements

Figure 2: Enhanced Relational Mapping



The relational mapping was updated according to the enhanced ERD. New tables and foreign key relationships were introduced while maintaining database normalization and referential integrity.

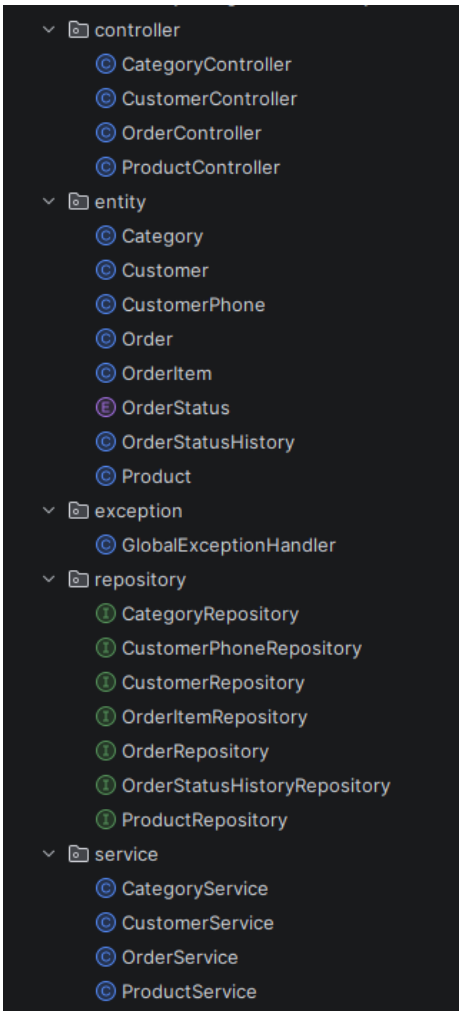
Following the ERD redesign, the relational mapping was updated accordingly.

The mapping improvements included:

- Mapping Customer's complex name into separate database columns.
- Creating a dedicated table for customer phone numbers.
- Creating a separate table for order status history.
- Updating foreign key relationships based on the enhanced ERD.
- Improving normalization by separating multivalued attributes into independent tables.

2.3 Application Enhancements

Figure 3: Application Package Structure



The project follows a layered architecture consisting of controller, service, repository, entity, and exception packages. This structure separates responsibilities and improves maintainability.

The application implementation was enhanced to support the redesigned database model and the newly introduced business requirements.

Four new classes were added to the project:

- **CustomerPhone** – Represents customer phone numbers and supports the multivalued phone number attribute through a one-to-many relationship with the Customer entity.
- **OrderStatusHistory** – Stores the history of order status changes, enabling the system to track the complete lifecycle of an order.
- **CustomerPhoneRepository** – Provides data access operations for the CustomerPhone entity using Spring Data JPA.
- **OrderStatusHistoryRepository** – Provides data access operations for the OrderStatusHistory entity and includes a derived query method for retrieving the status history of a specific order.

Several existing classes were also modified to integrate the new functionality:

- **Customer** – Replaced the single **name** attribute with **firstName** and **lastName**, and added a one-to-many relationship with CustomerPhone.
- **CustomerService** – Updated to support customer creation using firstName and lastName, injected CustomerPhoneRepository, and implemented business logic for adding customer phone numbers.
- **CustomerController** – Modified to accept firstName and lastName through CustomerRequest, introduced PhoneRequest, and added a new REST endpoint for adding customer phone numbers.
- **OrderService** – Extended by injecting OrderStatusHistoryRepository, implementing a reusable helper method for recording order status changes, and adding functionality to retrieve order history.
- **OrderController** – Added a new REST API endpoint for retrieving the status history of a specific order.
- **OrderStatusHistoryRepository** – Extended with a derived query method to retrieve the history records associated with a given order.

In addition, bidirectional entity relationships were updated using Jackson annotations (@JsonManagedReference, @JsonBackReference, and @JsonIgnore) to eliminate circular JSON serialization issues while maintaining correct object relationships and clean REST API responses.

These enhancements improved the application's maintainability, data integrity, and functionality while preserving the existing architecture of the original project.

The implemented enhancements were functionally verified through a comprehensive set of REST API test cases. The complete testing results are presented in **Appendix A.**

3. Challenges Faced While Working on the Project

One of the primary challenges was enhancing an existing project without affecting its original functionality. Since the project was already implemented, every modification required careful integration with the existing architecture.

Another challenge involved redesigning the database while maintaining consistency between the ERD, relational mapping, JPA entities, and REST API implementation. Introducing new entities such as **CustomerPhone** and **OrderStatusHistory** required updating multiple application layers simultaneously.

Implementing automatic order status history tracking was another challenge. Every status change had to be recorded consistently whenever an order was confirmed, cancelled, or updated without duplicating business logic.

Finally, extending entity relationships introduced JSON serialization problems caused by circular references between related entities.

4. Issues and Bugs Identified

The review of the original project identified several limitations that affected both the database design and the application functionality.

The Customer Entity stored the customer's full name as a single attribute, limiting flexibility when managing customer information. In addition, the system did not support multiple phone numbers for a single customer, making it difficult to represent real-world customer data.

The application also lacked a mechanism to record the history of order status changes. As a result, users could not track the complete lifecycle of an order after it was created.

The successful implementation of this enhancement was verified through the Order Status History API **(see Appendix A, Test Case A.8)**.

Furthermore, the original ERD required refinement to better represent the business requirements. Relationships and participation constraints needed improvement, and the relational mapping had to be updated to remain consistent with the enhanced database design.

Finally, bidirectional relationships between entities produced circular JSON serialization issues during REST API responses, resulting in serialization conflicts that required additional handling.

5. How the Issues Were Resolved

Figure 4: CustomerPhone Entity Implementation

```java
@Data
@Entity
@Table(name = "customers_phone")
public class CustomerPhone {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore
    @ManyToOne
    @JoinColumn(name = "Cus_id", nullable = false)
    private Customer customer;

    @NotBlank
    @Column(name = "phone_Number")
    private String phoneNumber;
}
```

The CustomerPhone entity implements the multivalued phone number feature by establishing a one-to-many relationship between customers and their phone numbers.

Figure 5: OrderStatusHistory Entity Implementation

```java
@Data
@Entity
@Table(name = "order_status_history")
public class OrderStatusHistory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "history_id")
    private Long historyId;

    @ManyToOne
    @JoinColumn(name = "Ord_id", nullable = false)
    private Order order;

    @Column(name = "old_status")
    private String oldStatus;

    @Column(name = "new_status")
    private String newStatus;

    @Column(name = "changed_at")
    private LocalDateTime changedAt = LocalDateTime.now();
}
```

The OrderStatusHistory entity records every order status transition, enabling complete tracking of the order lifecycle.

The project was enhanced by redesigning both the conceptual and physical database models.

The ERD was updated to include complex attributes, multivalued attributes, new entities, and improved relationships. The relational mapping was then redesigned to reflect these conceptual changes.

Two new entity classes were implemented:

- CustomerPhone
- OrderStatusHistory

Additionally, two new repository classes were created:

- CustomerPhoneRepository
- OrderStatusHistoryRepository

Several existing classes were also modified to integrate the new functionality. The Customer entity was redesigned by replacing the single **name** field with **firstName** and **lastName**, while introducing a one-to-many relationship with CustomerPhone.

The CustomerService and CustomerController classes were extended to support customer phone management, including adding phone numbers through a new REST API endpoint.

The customer creation and phone number management features were successfully validated through API testing **(see Appendix A, Test Cases A.1–A.2)**.

The OrderService was enhanced by introducing a reusable helper method to automatically record every order status transition whenever an order is confirmed, cancelled, or updated. It was also extended with functionality to retrieve the status history of a specific order. Correspondingly, the OrderController was updated by adding a new endpoint for retrieving order status history.

The enhanced order management functionality, including order creation, confirmation, shipping, cancellation, and status history tracking, was verified through functional API testing **(see Appendix A, Test Cases A.5–A.10)**.

Finally, JSON serialization issues caused by bidirectional entity relationships were resolved using the Jackson annotations **@JsonManagedReference**, **@JsonBackReference**, and **@JsonIgnore**. These annotations successfully eliminated circular references while preserving the required REST API response structure.

Additional validation was performed to verify inventory protection and reporting features, including oversell prevention and low-stock reporting **(see Appendix A, Test Cases A.11–A.12)**.

6. Technical Decisions Made by the Team

Several important technical decisions were made during the enhancement process.

The ERD was redesigned before modifying the implementation to ensure consistency between the conceptual model and the application.

Dedicated entities were created for customer phone numbers and order status history instead of embedding this information inside existing entities. This improved database normalization and scalability.

The Customer entity was redesigned to separate first and last names, providing greater flexibility for future development.

Spring Data JPA repositories were used to simplify database access through derived query methods instead of custom SQL queries.

Order history recording was centralized in a reusable helper method to avoid duplicated business logic.

REST API endpoints were extended while preserving the existing application architecture.

Jackson annotations were applied to resolve circular JSON serialization issues without changing the entity relationships.

Selected entity relationships were configured with eager fetching to provide complete nested JSON responses required by the API.

7. Suggestions for Improving the Original Project

Although the enhanced project significantly improves the original implementation, several additional improvements could further increase software quality.

Future enhancements may include:

- Implement comprehensive unit and integration testing.
- Add Bean Validation annotations for request validation.
- Generate API documentation using Swagger/OpenAPI.
- Improve global exception handling with standardized API error responses.
- Implement authentication and authorization using Spring Security.
- Add pagination and filtering for large datasets.
- Review fetch strategies to optimize performance for large-scale applications.

8. Lessons Learned from the Enhancement Exercise

This enhancement project provided valuable experience in maintaining and extending an existing software system rather than developing one from scratch.

The project reinforced the importance of designing a complete and well-structured ERD before implementation and maintaining consistency between conceptual models, relational mapping, and application code.

It also strengthened practical knowledge of Spring Boot, Spring Data JPA, RESTful API development, entity relationships, repository design, and JSON serialization management.

Furthermore, the project demonstrated the value of reusable business logic, database normalization, and layered software architecture in developing scalable and maintainable enterprise applications.

Overall, the exercise improved both technical skills and software engineering practices while emphasizing the importance of systematic analysis before implementing enhancements.

9. Conclusion

The enhancement of the Inventory & Order Management API successfully addressed several limitations identified in the original project. Improvements were made to the database design, relational mapping, application architecture, and REST API functionality. New entities, repositories, services, and endpoints were introduced to support additional business requirements while preserving the existing system architecture.

The final solution provides a more normalized database, improved maintainability, enhanced functionality, and a stronger foundation for future development.

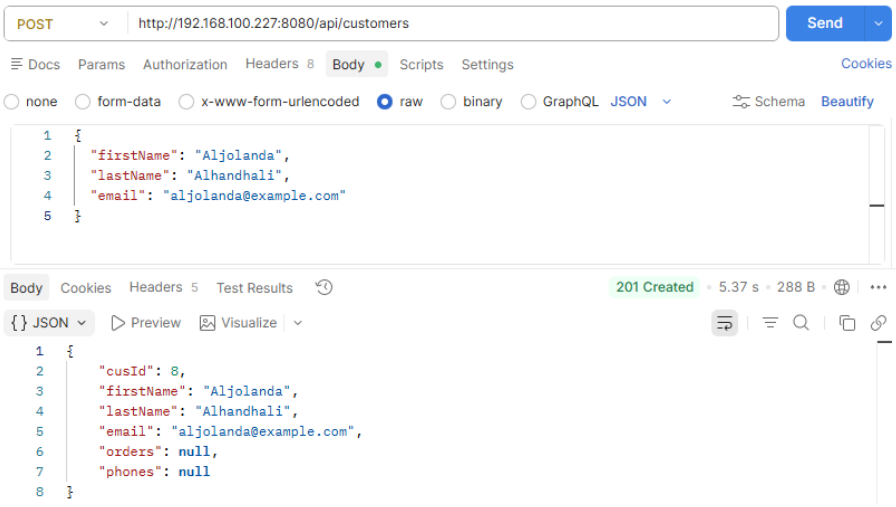
10. Appendix

Appendix A – API Functional Test Cases

A.1 Test 1 — Create Customer (with first & last name):

Objective: Verify that a new customer can be created using separate **firstName** and **lastName** fields.

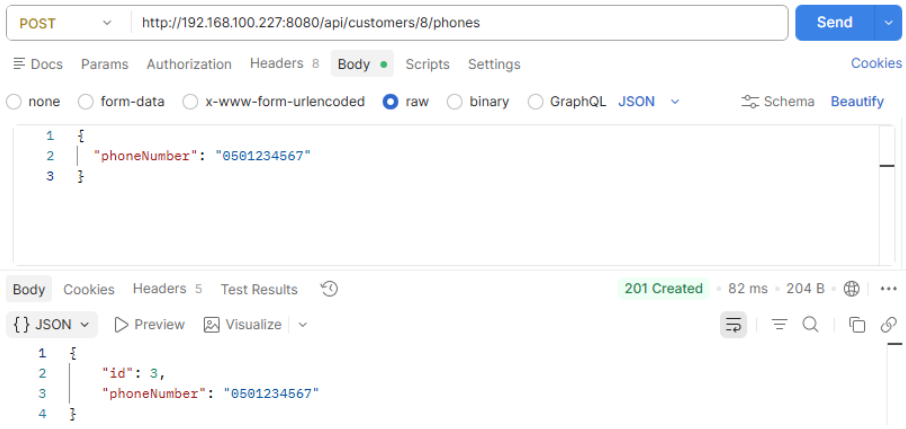
Expected Result:
The customer is successfully created and stored in the database.



A.2 Test 2 — Add Phone Number to Customer:

Objective: Verify that multiple phone numbers can be associated with a customer.

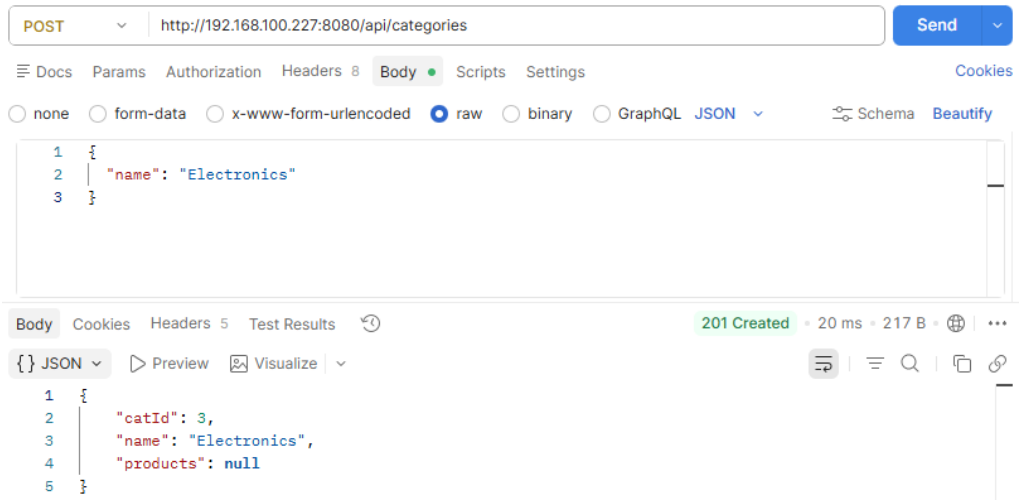
Expected Result:
The phone number is successfully added and linked to the selected customer.



A.3 Test 3 — Create Category:

Objective: Verify that a new product category can be created.

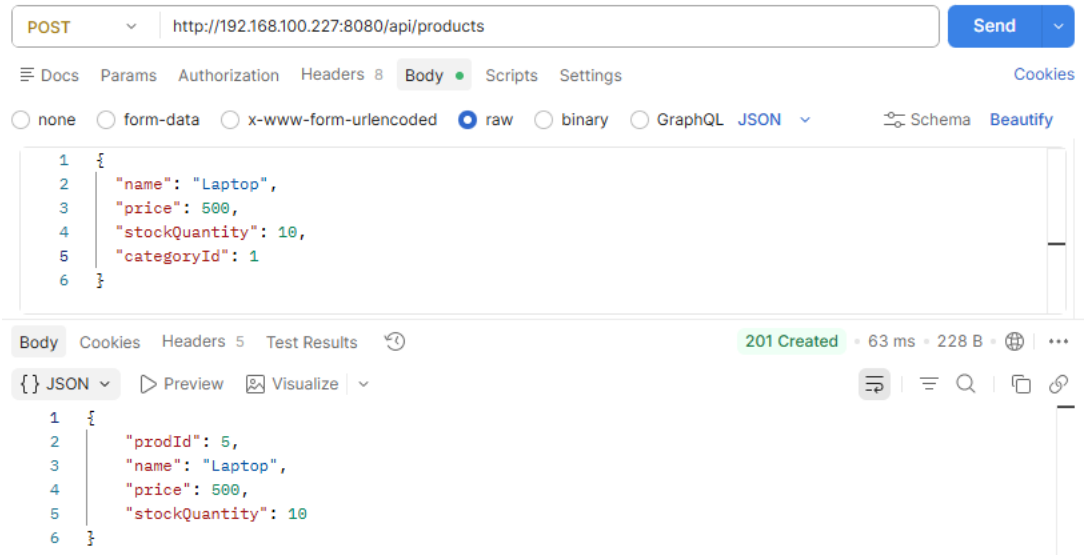
Expected Result:
The category is successfully stored in the database.



A.4 Test 4 — Create Product:

Objective: Verify that a new product can be created under an existing category.

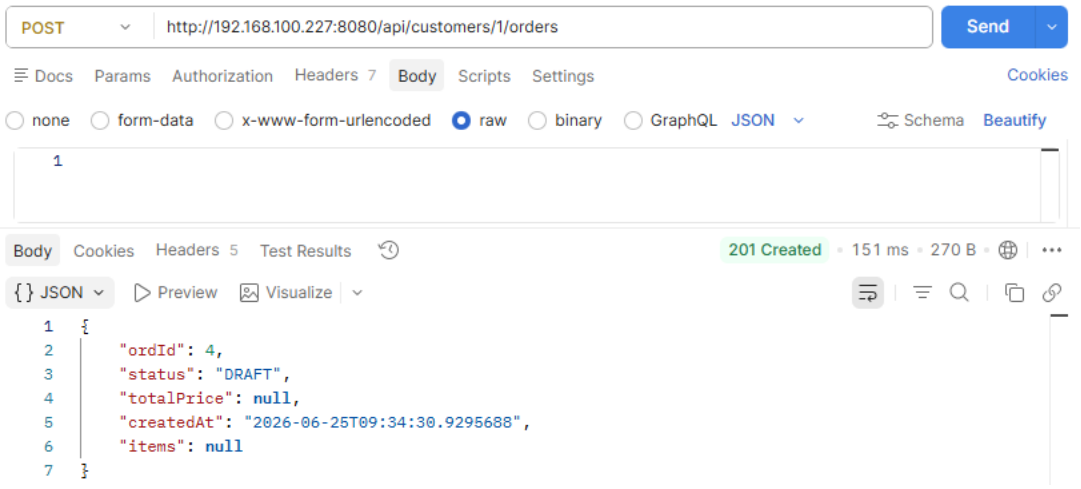
Expected Result:
The product is successfully created and associated with its category.



A 5 Test 5 — Create Draft Order:

Objective: Verify that a draft order can be created successfully.

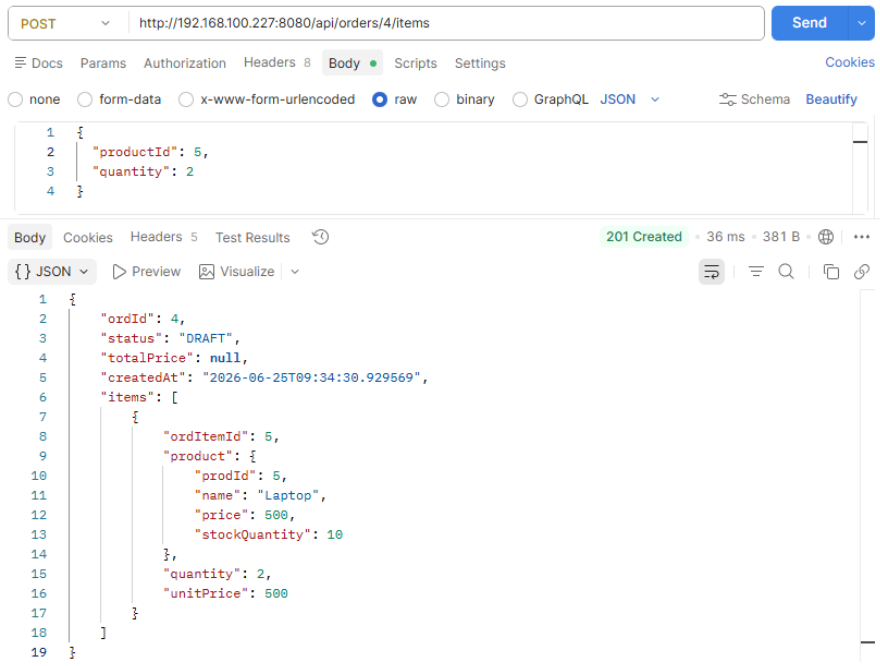
Expected Result:
A new order is created with the DRAFT status.



A.6 Test 6 — Add Item to Order:

Objective: Verify that products can be added to a draft order.

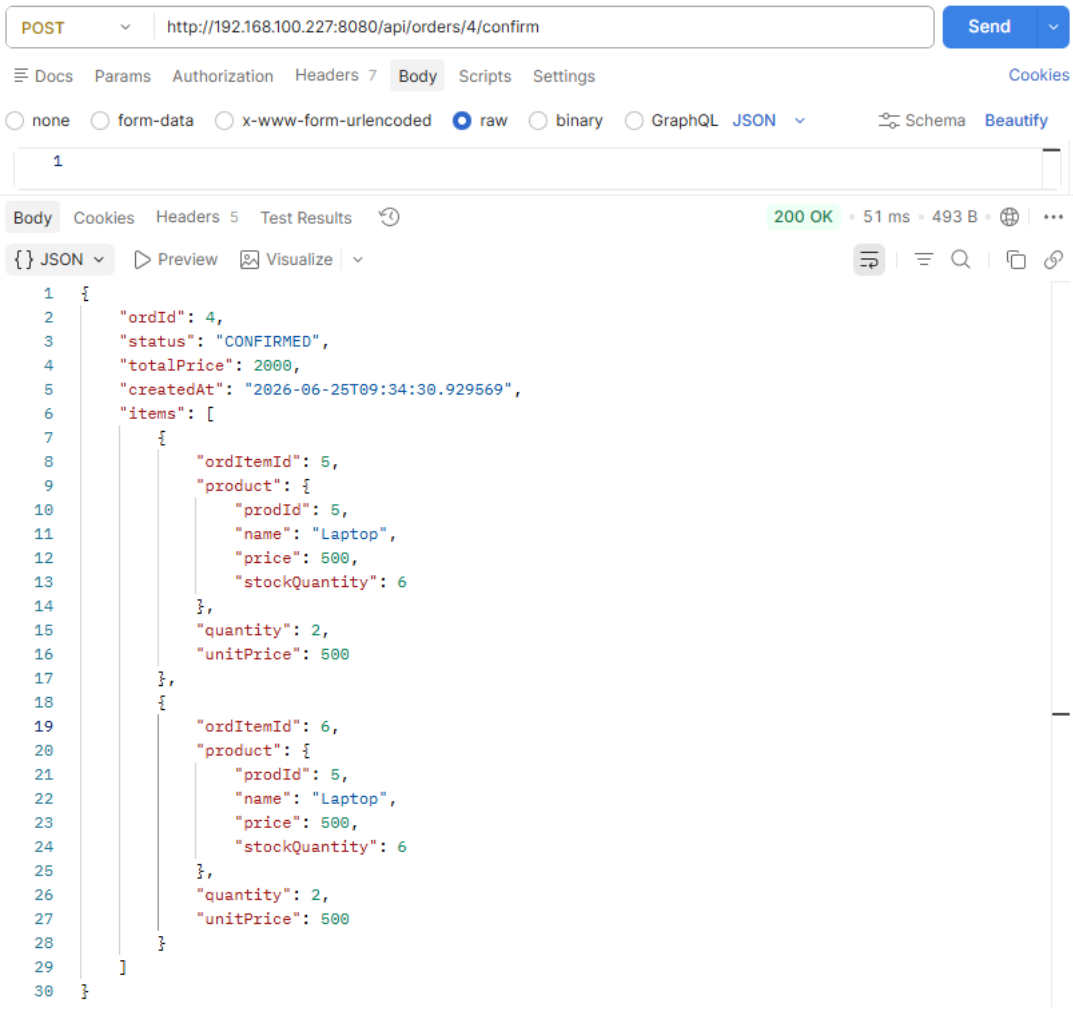
Expected Result:
The selected products are successfully added to the order.



A.7 Test 7 — Confirm Order:

Objective: Verify that an order can be confirmed successfully.

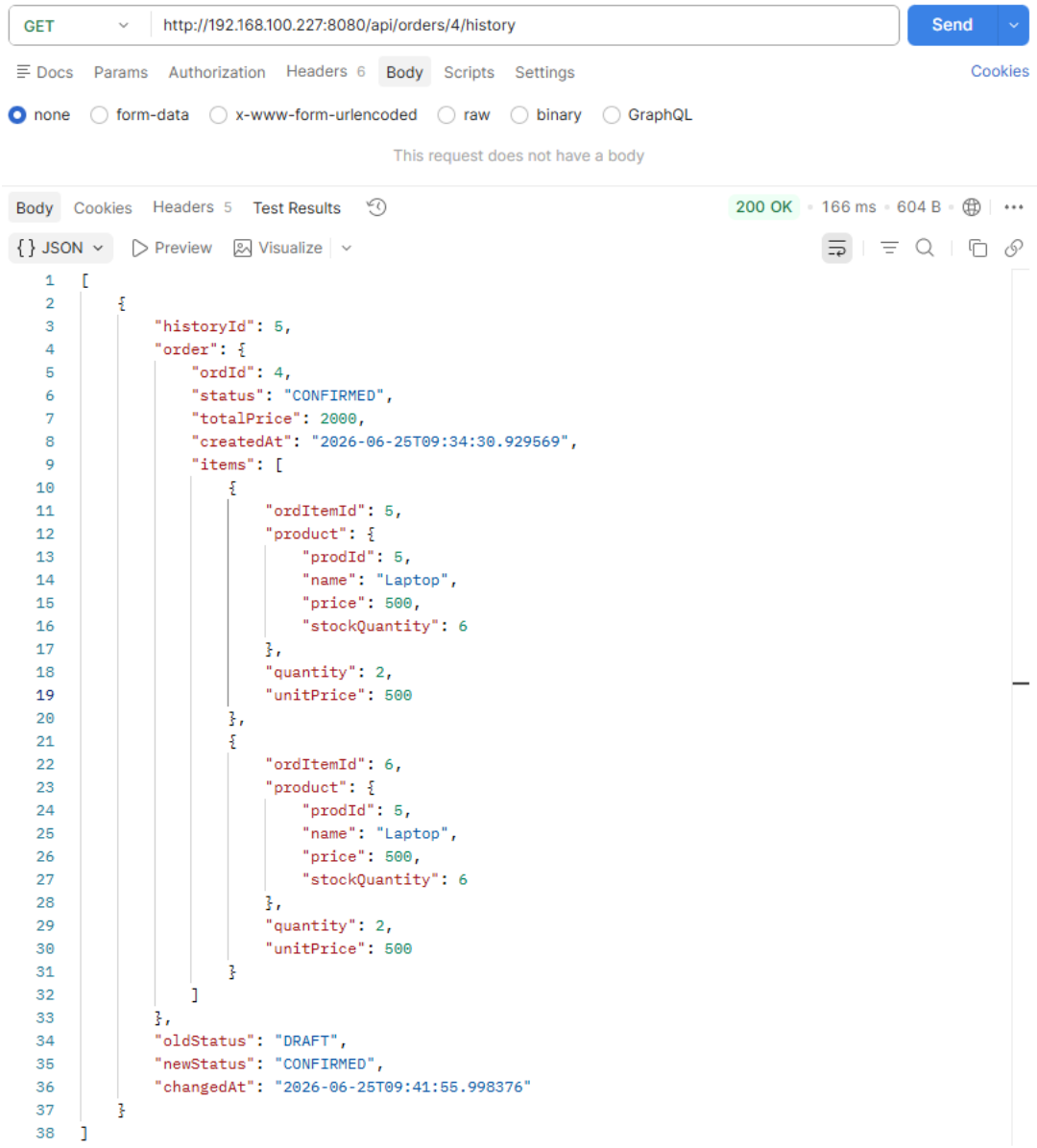
Expected Result:
The order status changes from **DRAFT** to **CONFIRMED**, and inventory quantities are updated.



A.8 Test 8 — Check Status History (DRAFT → CONFIRMED logged):

Objective: Verify that every order status change is automatically recorded.

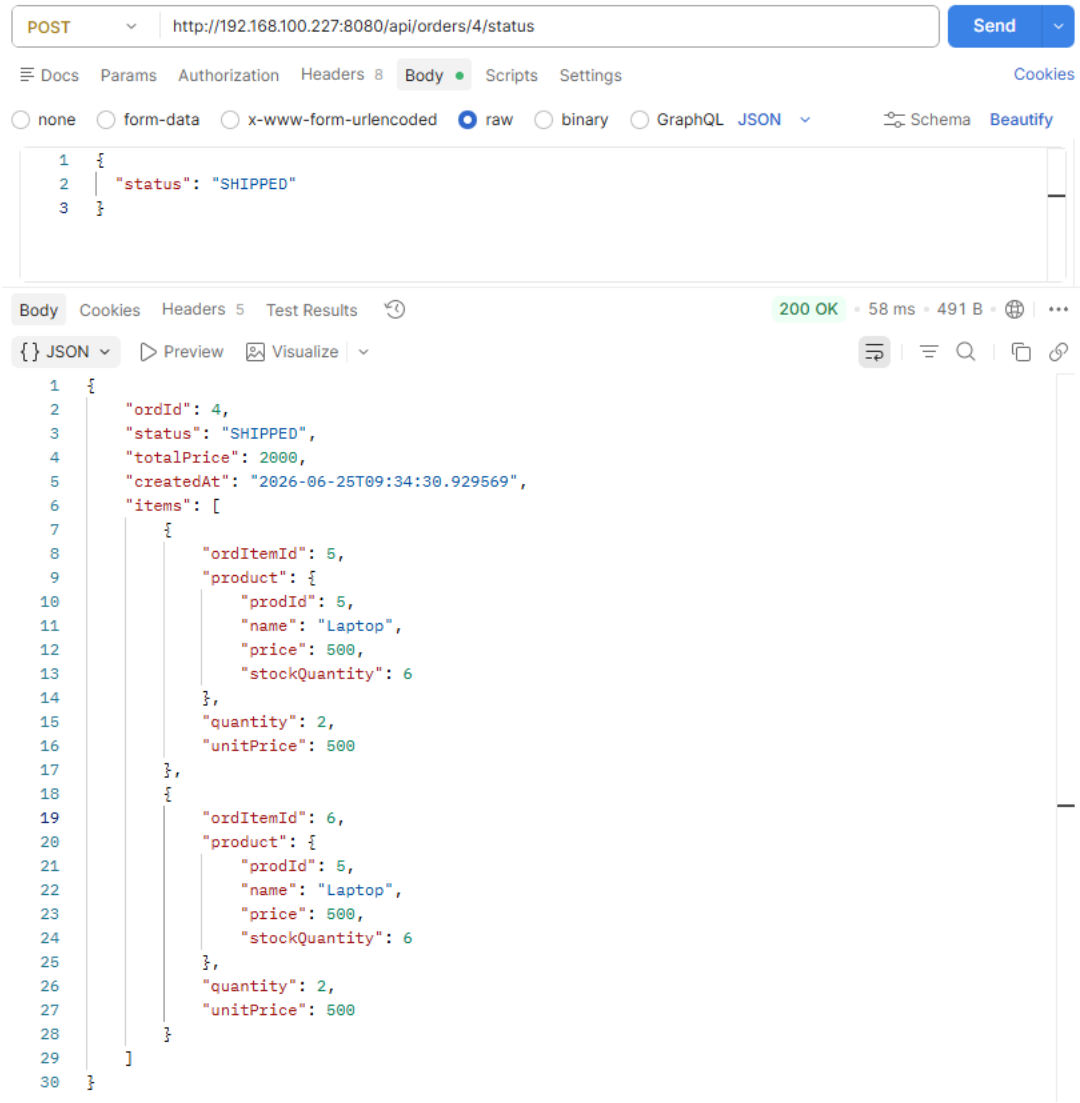
Expected Result:
The API returns the complete status history of the selected order.



A.9 Test 9 — Ship Order:

Objective: Verify that a confirmed order can be marked as shipped.

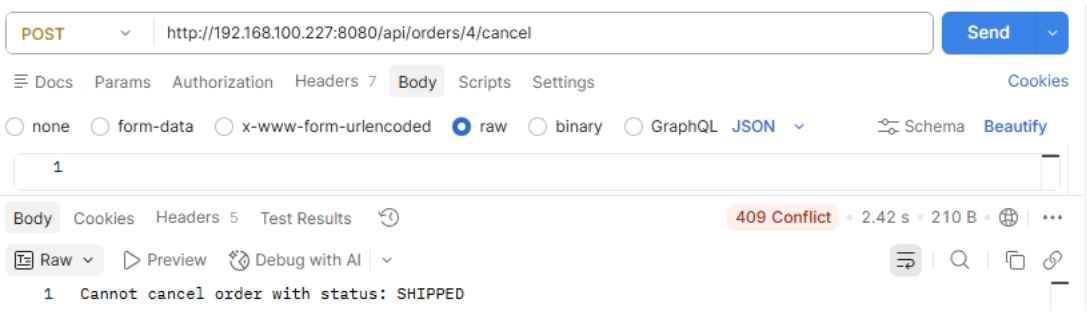
Expected Result:
The order status changes to **SHIPPED**, and the status history is updated.



A.10 Test 10 — Cancel Order (stock restored):

Objective: Verify that cancelling an order restores product stock quantities.

Expected Result:
The order status changes to **CANCELLED**, and inventory is restored correctly.



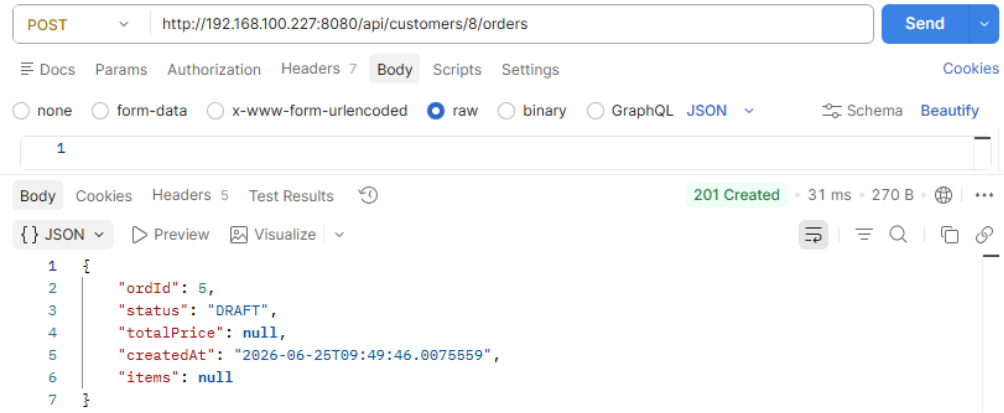
A.11 Test 11 — Oversell check:

Objective: Verify that the system prevents confirming an order when the requested quantity exceeds the available stock.

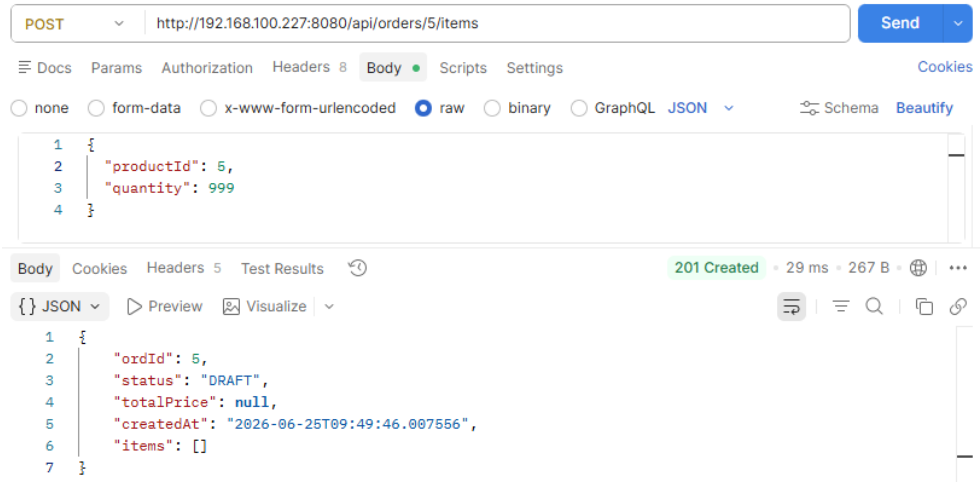
Expected Result:

- The confirmation request is rejected.
- The order remains DRAFT status.
- Product stock quantities remain unchanged.

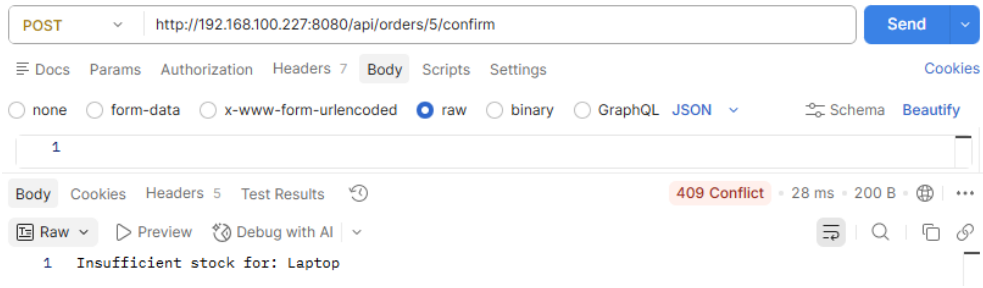
Step 1 — Create a new draft order:



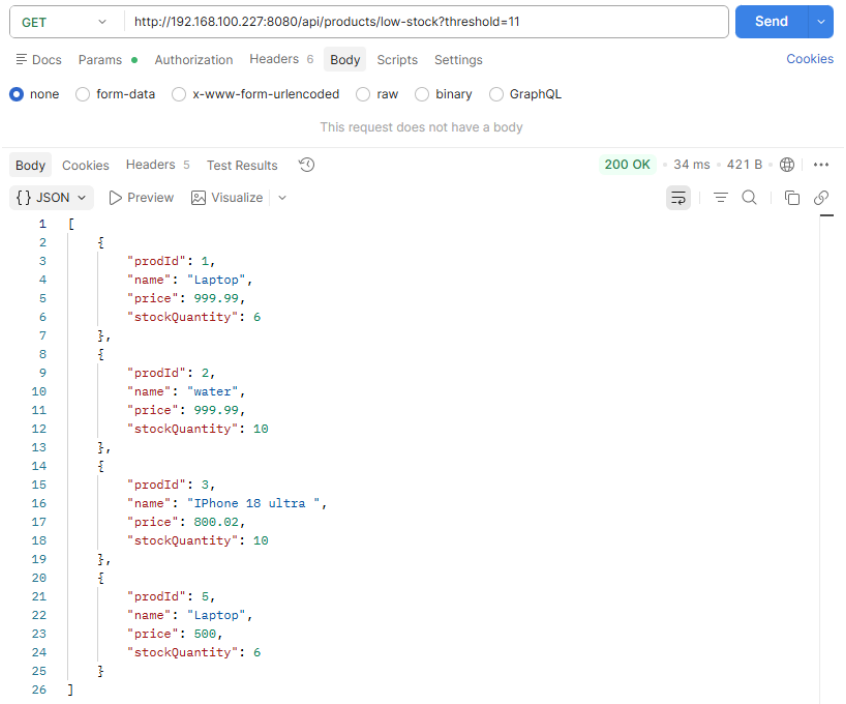
Step 2 — Add 999 laptops to the order:



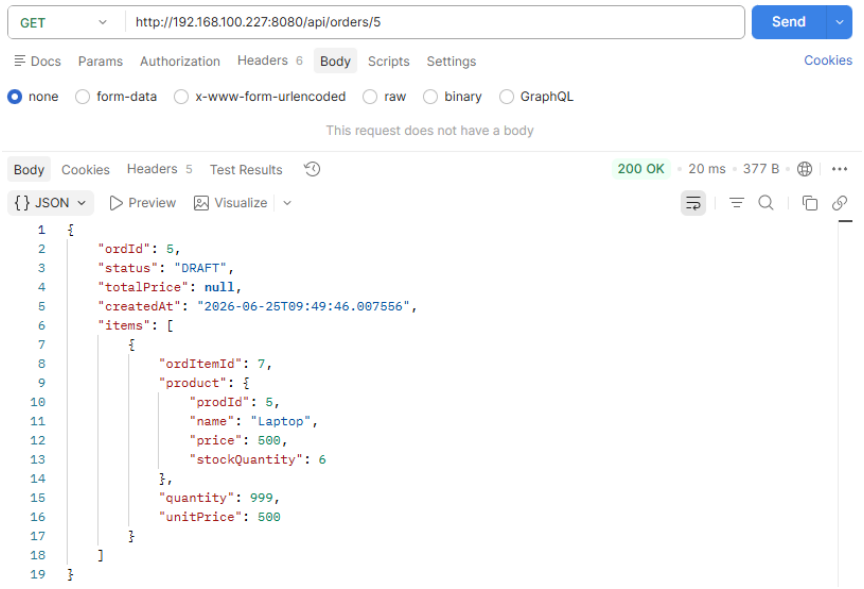
Step 3 — Try to confirm (THIS SHOULD FAIL):



Step 4 — Verify stock is still 6 (not touched):



Step 5 — Verify order is still DRAFT (not changed):



A.12 Test 12 — Low Stock Report:

Objective: Verify that the system generates an accurate low-stock report.

Expected Result:
The API returns all products whose stock quantity is below the configured threshold.